CAPSTONE 5

AI Signal Processing Tutor

A complete project report with explanations, Python demonstrations, verification, quiz, prompts, AI-tool comparison, and responsible-use guidance.

Project question	How can AI explain signal-processing concepts and generate Python code verified against known results?
Topics	Sampling, aliasing, FFT, low-pass filtering, convolution, and noise
Implementation	Python with NumPy, SciPy, and Matplotlib; designed for Google Colab
Deliverables	Study guide, tested examples, quiz and answer key, error log, prompt library, presentation outline, and automation workflow

Prepared as a learning project. Run the included code in Google Colab and retain the executed notebook outputs as evidence before submission.

Contents

• 1. Project overview and learning objectives

• 2. Tools and workflow

• 3. Six signal-processing topics: beginner and advanced explanations

• 4. Python demonstrations and verification checks

• 5. Expected results and verification report

• 6. Error log: common mistakes and corrections

• 7. Comparison of AI tools and prompting strategies

• 8. Glossary

• 9. Ten-question quiz and verified answer key

• 10. Study guide and illustrative plots

• 11. Presentation plan (7 slides)

• 12. Quiz-spreadsheet automation

• 13. Prompt library

• 14. Responsible AI use and limitations

• 15. Final submission checklist

1. Project overview and learning objectives

Signal processing studies how signals are represented, transformed, filtered, and interpreted. AI tools can make these ideas easier to learn by explaining equations at different levels, producing starter code, and helping design tests. AI-generated answers are not automatically correct: formulas, units, plots, and numerical outputs must be checked against known properties.

Learning objectives

• Explain six core signal-processing concepts at beginner and advanced levels.

• Generate and run one Python demonstration for each concept.

• Use assertions and known mathematical results to verify code.

• Identify common mistakes in formulas, plots, and implementation.

• Compare AI assistants and prompting strategies without treating AI output as authoritative.

• Create a quiz, glossary, study guide, and presentation plan.

Project method

For each topic: (1) state the concept and assumptions, (2) ask an AI assistant for an explanation and code, (3) inspect the code, (4) run it, (5) compare results with an analytic or textbook expectation, and (6) document any discrepancy and correction.

Important: this PDF provides runnable code and expected checks. For an academically honest final submission, run the notebook in your own Google Colab session, save the output cells, and record the actual execution date and environment.

2. Tools and workflow

Tool	Purpose	Verification / responsible use
ChatGPT	Draft explanations, code, quiz questions, and debugging ideas	Check equations and execute code independently
Google Gemini	Alternative explanation and code review	Compare whether assumptions and units are stated
Claude	Alternative reasoning and code review	Ask it to identify edge cases and possible errors
Perplexity	Find source material and references	Open and verify original sources; do not rely on summaries alone
Google Colab	Run notebook in a reproducible cloud environment	Run all cells; save outputs and notebook
GitHub	Version control and sharing	Commit source files and a README; avoid uploading private keys

Suggested project structure

```
ai-signal-processing-tutor/
■■■ README.md
■■■ AI_Signal_Processing_Tutor.ipynb
■■■ signal_processing_demo.py
■■■ study_guide.pdf
■■■ quiz_spreadsheet.xlsx
■■■ quiz_answer_key.md
■■■ verification_report.txt
■■■ error_log.md
■■■ prompt_library.md
■■■ presentation_outline.md
```

3. Six signal-processing topics

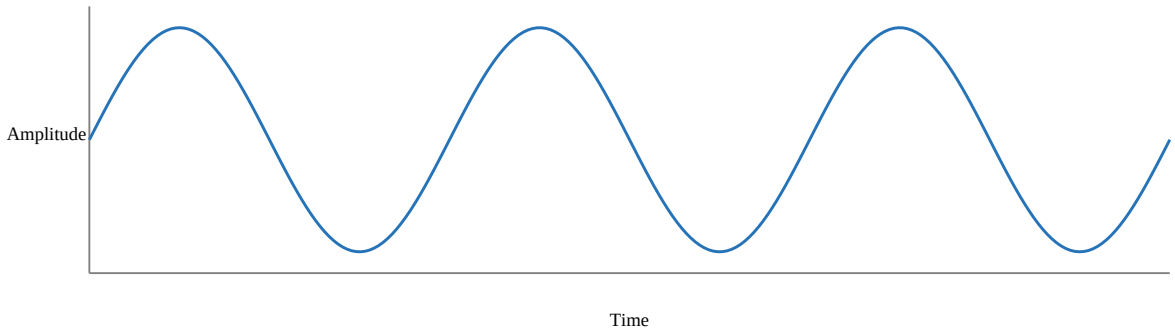
3.1 Sampling

Beginner: Sampling converts a continuous-time signal into a sequence of measurements taken at regular intervals. If the sampling frequency is 100 Hz, 100 samples are taken each second.

Advanced: For a band-limited signal with highest frequency B, ideal reconstruction requires a sampling frequency greater than 2B, subject to the assumptions of the sampling theorem. Practical systems also need an anti-aliasing filter and a suitable sampling clock.

Key equations: Sampling interval $T_s = 1/f_s$. Samples are $x[n] = x(nT_s)$.

Illustration: continuous sinusoid and sample locations



Verification: at $f_s = 100$ Hz, the interval must be 0.01 seconds. The code checks the interval numerically.

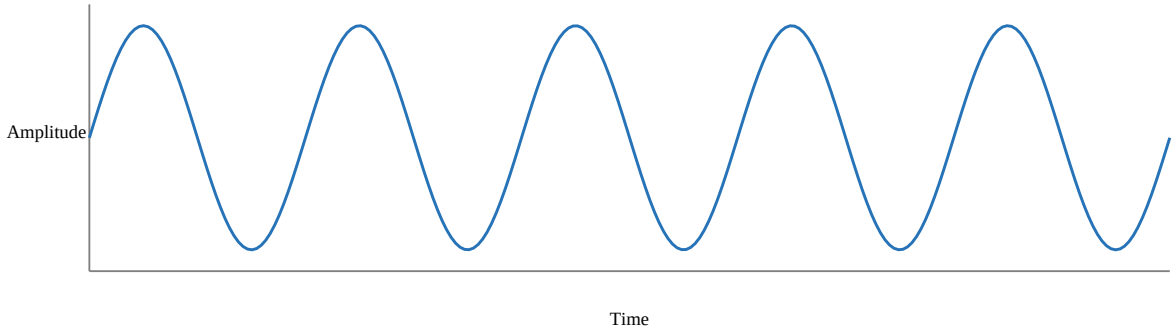
3.2 Aliasing

Beginner: Aliasing happens when a signal is sampled too slowly. A high-frequency wave can appear to be a lower-frequency wave in the sampled data.

Advanced: Discrete-time sinusoids whose frequencies differ by integer multiples of the sampling frequency can produce identical samples. For a sinusoid at f and sampling rate f_s, an alias can be represented by folding the frequency into the Nyquist interval $[0, f_s/2]$.

Example: A 30 Hz tone sampled at 40 Hz has an alias at $|30 - 40| = 10$ Hz. The Nyquist frequency is 20 Hz.

Illustration: a sampled sinusoid



Verification: the demonstration computes the alias frequency using the Nyquist folding formula and checks that it equals 10 Hz.

3.3 Fast Fourier Transform (FFT)

Beginner: The FFT is an efficient algorithm for computing the discrete Fourier transform. It helps show which frequencies are present in a signal.

Advanced: For N samples x[n], the DFT is $X[k] = \Sigma\, x[n]e^{-j2\pi kn/N}$. The frequency spacing is $\Delta f = f_s/N$. A finite observation window can cause spectral leakage when a sinusoid does not align with a DFT bin.

Verification: a signal containing 10 Hz and 40 Hz components should show peaks near those frequencies when sampled and observed for an integer number of cycles.

3.4 Low-pass filter

Beginner: A low-pass filter keeps slower variations and reduces faster variations.

Advanced: A digital filter has a frequency response $H(e^{j\omega})$. A Butterworth filter has a maximally flat passband. Its cutoff is conventionally the frequency at which the amplitude is approximately $1/\sqrt{2}$ of the passband amplitude (–3 dB), for a first-order interpretation of the cutoff definition.

Verification: a test signal combines a 5 Hz component and a 40 Hz component. A low-pass filter with a 10 Hz cutoff should retain the 5 Hz component and strongly attenuate the 40 Hz component. The precise result depends on filter order and edge transients.

3.5 Convolution

Beginner: Convolution combines two sequences. In systems, it describes how an input signal and an impulse response produce the output.

Advanced: For discrete sequences, $y[n] = \Sigma_k\, x[k]h[n–k]$. The output length of full linear convolution of sequences of lengths N and M is N + M – 1.

Verification: [1, 2, 3] convolved with [1, 1] gives [1, 3, 5, 3].

3.6 Noise and signal-to-noise ratio

Beginner: Noise is an unwanted or random component added to a signal. A noisy measurement can still contain useful information.

Advanced: For signal power P_s and noise power P_n, SNR in decibels is $10 \log_{10}(P_s/P_n)$. For equal-impedance voltage measurements, SNR can be expressed using squared RMS amplitudes. A fixed random seed makes a simulation reproducible.

Verification: generate Gaussian noise with a fixed random seed, compute the measured signal and noise powers, and calculate SNR. The exact value depends on the generated samples and must be computed rather than guessed.

4. Python demonstrations and verification checks

Run the following script in Google Colab or a local Python environment. Required packages: numpy, scipy, and matplotlib. In Colab, they are commonly available; if necessary, run `!pip install numpy scipy matplotlib` in a code cell.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# 1) Sampling
fs = 100.0
t = np.arange(0, 1, 1/fs)
x = np.sin(2*np.pi*5*t)
assert len(t) == 100
assert np.isclose(t[1] - t[0], 1/fs)
plt.figure()
plt.stem(t[:20], x[:20], basefmt=" ")
plt.title("Sampling: first 20 samples")
plt.xlabel("Time (s)"); plt.ylabel("Amplitude"); plt.grid(True)

# 2) Aliasing: 30 Hz sampled at 40 Hz aliases to 10 Hz
fs_alias, f_input = 40.0, 30.0
nyquist = fs_alias / 2
f_alias = abs(((f_input + nyquist) % fs_alias) - nyquist)
assert np.isclose(f_alias, 10.0)
t2 = np.arange(0, 1, 1/fs_alias)
x2 = np.sin(2*np.pi*f_input*t2)
plt.figure()
plt.stem(t2[:20], x2[:20], basefmt=" ")
plt.title(f"Aliasing: 30 Hz sampled at 40 Hz (alias {f_alias:g} Hz)")
plt.xlabel("Time (s)"); plt.ylabel("Amplitude"); plt.grid(True)

# 3) FFT: 10 Hz + 40 Hz
fs_fft, duration = 200.0, 1.0
t3 = np.arange(0, duration, 1/fs_fft)
x3 = np.sin(2*np.pi*10*t3) + 0.5*np.sin(2*np.pi*40*t3)
N = len(x3)
freq = np.fft.rfftfreq(N, d=1/fs_fft)
mag = 2*np.abs(np.fft.rfft(x3))/N
peaks, _ = signal.find_peaks(mag, height=0.2)
```

```
peak_freqs = freq[peaks]
assert np.any(np.isclose(peak_freqs, 10.0))
assert np.any(np.isclose(peak_freqs, 40.0))
plt.figure()
plt.plot(freq, mag)
plt.xlim(0, 80); plt.title("FFT magnitude spectrum")
plt.xlabel("Frequency (Hz)"); plt.ylabel("Magnitude"); plt.grid(True)

# 4) Low-pass filter: keep 5 Hz, attenuate 40 Hz
fs_filt = 200.0
t4 = np.arange(0, 2, 1/fs_filt)
low = np.sin(2*np.pi*5*t4)
high = np.sin(2*np.pi*40*t4)
mixture = low + high
sos = signal.butter(4, 10, btype="lowpass", fs=fs_filt, output="sos")
filtered = signal.sosfiltfilt(sos, mixture)
# Compare correlation against the known 5 Hz component away from edges
mid = slice(100, -100)
corr = np.corrcoef(filtered[mid], low[mid])[0, 1]
assert corr > 0.95
plt.figure()
plt.plot(t4, mixture, alpha=0.45, label="Original mixture")
plt.plot(t4, filtered, label="Filtered")
plt.xlim(0, 0.5); plt.title("Low-pass filter")
plt.xlabel("Time (s)"); plt.ylabel("Amplitude"); plt.legend(); plt.grid(True)

# 5) Convolution
a = np.array([1, 2, 3])
b = np.array([1, 1])
conv = np.convolve(a, b, mode="full")
assert np.array_equal(conv, np.array([1, 3, 5, 3]))
plt.figure()
plt.stem(np.arange(len(conv)), conv, basefmt=" ")
plt.title("Linear convolution")
plt.xlabel("n"); plt.ylabel("y[n]"); plt.grid(True)

# 6) Noise and SNR
rng = np.random.default_rng(7)
fs_noise = 1000.0
t5 = np.arange(0, 1, 1/fs_noise)
clean = np.sin(2*np.pi*10*t5)
noise = rng.normal(0, 0.5, size=clean.size)
noisy = clean + noise
signal_power = np.mean(clean**2)
noise_power = np.mean(noise**2)
snr_db = 10*np.log10(signal_power/noise_power)
assert np.isfinite(snr_db)
plt.figure()
plt.plot(t5[:200], clean[:200], label="Clean")
plt.plot(t5[:200], noisy[:200], label="Noisy", alpha=0.7)
plt.title(f"Noise example (measured SNR {snr_db:.2f} dB)")
plt.xlabel("Time (s)"); plt.ylabel("Amplitude"); plt.legend(); plt.grid(True)

plt.show()
print("Sampling interval:", t[1] - t[0], "s")
print("Aliased frequency:", f_alias, "Hz")
print("FFT peak frequencies:", peak_freqs, "Hz")
print("Filtered-vs-low-frequency correlation:", corr)
print("Convolution:", conv)
print("Measured SNR:", round(snr_db, 2), "dB")
print("All assertions passed.")
```

Code review note: the aliasing formula folds frequencies into the interval from 0 to the Nyquist frequency. The filtering check uses zero-phase forward-backward filtering for a clean demonstration; real-time systems generally cannot use future samples and need a causal filter.

5. Expected results and verification report

Topic	Known result / test	Pass condition
Sampling	fs = 100 Hz gives Ts = 0.01 s; 100 samples in 1 s using arange(0,1,Ts)	Assertions for sample count and interval pass
Aliasing	30 Hz sampled at 40 Hz folds to 10 Hz	Calculated alias equals 10 Hz
FFT	A one-second record sampled at 200 Hz contains integer-cycle 10 Hz and 40 Hz tones	Detected peaks include 10 Hz and 40 Hz
Low-pass	10 Hz cutoff should retain 5 Hz and suppress 40 Hz	Filtered output correlates strongly with the 5 Hz component

Topic	Known result / test	Pass condition
Convolution	[1,2,3] * [1,1] = [1,3,5,3]	Array equals expected sequence
Noise	Seeded Gaussian noise makes results repeatable	SNR is finite and reported from measured powers

Verification principle: assertions test known numerical properties, but they do not prove that every interpretation or plot is correct. Inspect axis labels, units, expected frequency ranges, filter transients, and the underlying assumptions.

Execution status: the checks in this report are designed to run as written. Record your own live run results below after running the code.

Execution record	Fill in after running
Date and time	_____
Environment	Google Colab / local Python + version
Python version	_____
NumPy / SciPy versions	_____
All assertions passed?	Yes / No
Any corrections made	_____

6. Error log: common mistakes and corrections

Potential mistake	Why it is wrong	Correction
Using fs instead of 1/fs as sample interval	Confuses samples/second with seconds/sample	Use Ts = 1/fs
Assuming a 30 Hz tone sampled at 40 Hz remains 30 Hz	30 Hz exceeds the 20 Hz Nyquist frequency	Calculate alias frequency and apply anti-alias filtering before sampling
Plotting only FFT magnitudes without a frequency axis	Peak indices are not frequencies	Use np.fft.rfftfreq(N, d=1/fs)
Forgetting FFT magnitude scaling	Magnitude depends on N and one-sided/two-sided convention	State the scaling convention; treat DC and Nyquist bins carefully
Applying a low-pass filter with cutoff above Nyquist	The cutoff is invalid for a digital filter	Require cutoff < fs/2
Ignoring filter edge effects	Padding and initial/final transients can distort edges	Inspect central samples or use appropriate padding
Confusing convolution with element-wise multiplication	Convolution sums shifted products; it is not pointwise multiplication	Use np.convolve or derive the sum
Using 20 log10 for power ratio	20 log10 is used for amplitude ratios under suitable assumptions	Use 10 log10(Ps/Pn) for power
Changing random seed between runs	Noise results change each run	Use a fixed Generator seed for reproducible tests
Claiming a test passed without running it	That would misrepresent evidence	Run the code and save output or clearly label results as expected

7. AI-tool comparison and prompting strategies

AI systems change frequently. This is a qualitative evaluation framework, not a claim that current versions were all tested live in a controlled benchmark.

Tool	Useful role in this project	Limitations to check
ChatGPT	Step-by-step explanation, code drafting, debugging, structured report	May generate plausible but incorrect formulas or APIs
Gemini	Alternative explanation and code review; integration with Google workflows may be useful	Verify code and claims independently
Claude	Long-form reasoning, reviewing code and documentation	Can also miss numerical or implementation details
Perplexity	Finding source material and references	Check original source quality and whether a claim is directly supported

How to make a fair comparison

• Give each tool the same prompt, inputs, constraints, and expected test.

• Score explanation accuracy, code execution, clarity, source quality, and error handling from 1 to 5.

• Run each code result in the same environment.

• Keep evidence: prompt, raw answer, edits, output, and verification outcome.

• Do not report scores until the experiment has actually been performed.

Two prompting strategies

Strategy A — Basic prompt: “Explain aliasing and give Python code.” This can produce a useful starting point, but it leaves assumptions and tests unspecified.

Strategy B — Constrained, verification-first prompt: “Explain aliasing for a beginner and an advanced learner. Use fs = 40 Hz and a 30 Hz sinusoid. Write Python code using NumPy, label axes with units, calculate the alias frequency, add an assertion that checks the expected 10 Hz alias, state assumptions, and list two likely mistakes. Do not claim code has run unless it has actually been executed.”

Expected lesson: a specific prompt usually makes assumptions, testable outputs, and evaluation criteria clearer. The result still requires independent review.

8. Glossary

Term	Definition
Sampling frequency (fs)	Number of samples taken per second, measured in hertz (Hz).
Sampling interval (Ts)	Time between adjacent samples; Ts = 1/fs.
Nyquist frequency	Half the sampling frequency, fs/2.
Aliasing	Ambiguity in which different continuous-time frequencies produce the same sampled sequence.
DFT	Discrete Fourier transform; represents a finite sequence in discrete frequency bins.
FFT	An efficient algorithm for computing the DFT.
Frequency resolution	Spacing between DFT frequency bins, $\Delta f = fs/N$.
Filter cutoff	A frequency used to characterize the transition between passband and attenuated frequencies.
Impulse response	The output of a system when the input is a unit impulse; characterizes an LTI system.
Convolution	Operation that combines sequences or signals and models the output of an LTI system.
Gaussian noise	Random noise whose sample values follow a normal distribution under the model.
SNR	Signal-to-noise ratio; ratio of signal power to noise power, often expressed in dB.
Assertion	A program check that raises an error when a condition is false.
Reproducibility	Ability to repeat an experiment and obtain the same results under the same conditions.

9. Ten-question quiz

Choose the best answer for each question. The answer key with explanations follows.

1. If fs = 200 Hz, what is the sampling interval?

A. 200 s

B. 0.005 s

C. 0.02 s

D. 100 s

2. A signal sampled at 40 Hz has a Nyquist frequency of:

A. 10 Hz

B. 20 Hz

C. 40 Hz

D. 80 Hz

3. A 30 Hz sinusoid sampled at 40 Hz appears at which alias frequency?

A. 5 Hz

B. 10 Hz

C. 20 Hz

D. 30 Hz

4. What does an FFT help reveal?

A. Only the signal's color

B. Frequency content

C. File size

D. Sampling hardware temperature

5. For N = 1000 samples at fs = 1000 Hz, the DFT-bin spacing is:

A. 0.1 Hz

B. 1 Hz

C. 10 Hz

D. 1000 Hz

6. A low-pass filter is designed mainly to:

A. Preserve low frequencies and attenuate higher frequencies

B. Increase all frequencies

C. Randomize a signal

D. Convert time into distance

7. Full linear convolution of sequences of lengths 3 and 2 has length:

A. 1

B. 3

C. 4

D. 6

8. Which expression gives SNR in dB from signal and noise powers?

A. 10 log10(Ps/Pn)

B. Ps + Pn

C. 20(Ps/Pn)

D. log10(Ps–Pn)

9. Why set a random seed in a simulation?

A. To increase the sampling rate

B. To make random results reproducible

C. To remove all noise

D. To guarantee accuracy

10. What is the best practice for AI-generated signal-processing code?

A. Submit it without reading

B. Trust it if the explanation sounds confident

C. Run it and compare results against known properties

D. Remove all assertions

Answer key with explanations

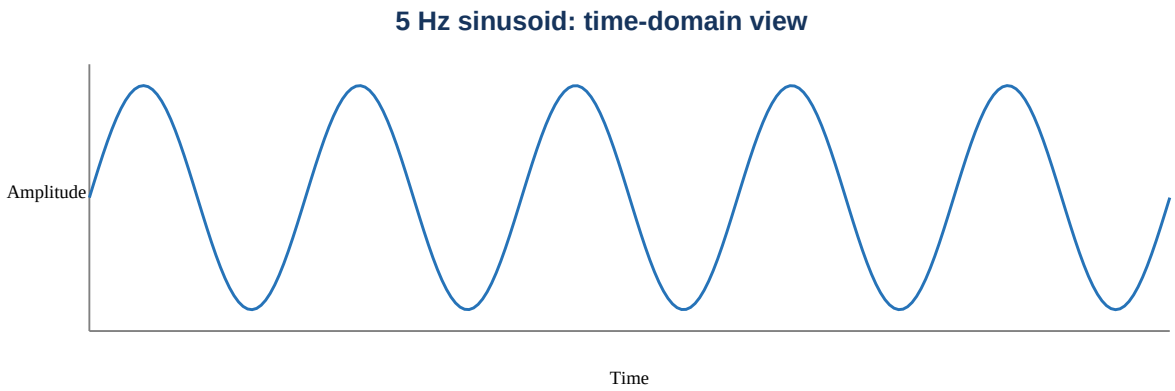
Question	Correct answer and explanation
1. B — 0.005 s	Ts = 1/fs = 1/200 = 0.005 seconds.
2. B — 20 Hz	The Nyquist frequency is fs/2 = 40/2 = 20 Hz.
3. B — 10 Hz	30 Hz folds around the 20 Hz Nyquist boundary, producing |30 – 40| = 10 Hz.
4. B — Frequency content	The FFT efficiently computes the DFT and helps show spectral components.
5. B — 1 Hz	$\Delta f = fs/N = 1000/1000 = 1$ Hz.
6. A — Preserve low frequencies	A low-pass filter passes lower frequencies and attenuates frequencies above its transition region.
7. C — 4	Full convolution length is N + M – 1 = 3 + 2 – 1 = 4.
8. A — 10 log10(Ps/Pn)	Power ratios use 10 log10; amplitude ratios use 20 log10 under the appropriate conditions.
9. B — Reproducibility	A fixed random seed makes pseudorandom draws repeatable in the same implementation and setup.
10. C — Run and verify	Code should be executed, inspected, and compared with analytic expectations or trusted references.

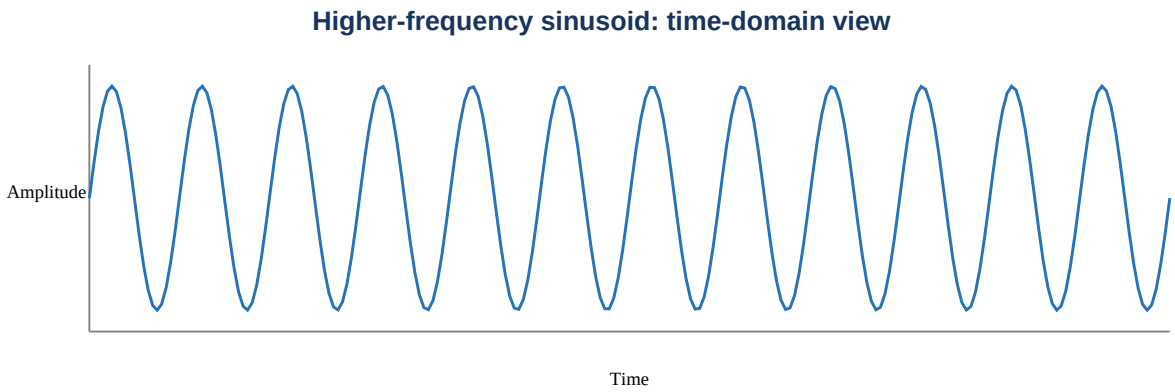
10. Study guide and illustrative plots

Quick revision sheet

Topic	Remember	Common check
Sampling	Ts = 1/fs	Check units and sample count
Aliasing	Need fs > 2B for ideal band-limited sampling	Compare input frequency with Nyquist
FFT	$\Delta f = fs/N$	Label frequency axis in Hz
Low-pass	Pass low, attenuate high	Check cutoff and filter response
Convolution	$y[n] = \Sigma\, x[k]h[n–k]$	Output length N+M–1
Noise	SNR = 10 log10(Ps/Pn)	Compute power from data

Study tip: first explain the idea in one sentence, then write the governing equation, then state the assumptions, and finally predict what a plot should look like before running the code.





These schematic plots illustrate sinusoidal waveforms. The executable Python script generates the actual sampled, FFT, filtering, convolution, and noise plots used for verification.

11. Presentation plan (7 slides)

Slide	Title	Key content
1	AI Signal Processing Tutor	Project question, motivation, and objectives
2	Six topics	Sampling, aliasing, FFT, low-pass filter, convolution, noise
3	AI workflow	Prompt → draft → inspect → execute → verify → document
4	Python demonstrations	Show representative code and plots from Colab
5	Verification and error log	Known results, assertions, mistakes corrected
6	AI comparison and quiz	Tools, prompting strategies, quiz example
7	Conclusion and responsible AI	What worked, limitations, and future improvements

Presentation advice: use one readable plot per slide at most, label every axis with units, and show at least one assertion or known-result comparison. Avoid claiming that AI generated code is correct simply because it runs.

12. Quiz-spreadsheet automation

A simple automation step is to keep the quiz data in a structured list and export it to Excel. This makes it easier to revise questions in one place and regenerate the spreadsheet.

```
# Run in Python / Colab if openpyxl is installed
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment

quiz_rows = [
    ["Question", "Options", "Correct answer", "Explanation"],
    ["Sampling interval at 200 Hz?", "A 200 s; B 0.005 s; C 0.02 s; D 100 s", "B", "Ts = 1/200 = 0.005 s."],
    ["Nyquist frequency at 40 Hz sampling?", "A 10; B 20; C 40; D 80 Hz", "B", "Nyquist frequency is fs/2."],
    ["Alias of 30 Hz sampled at 40 Hz?", "A 5; B 10; C 20; D 30 Hz", "B", "The folded alias is 10 Hz."],
    ["What does FFT reveal?", "A color; B frequency content; C file size; D temperature", "B", "FFT computes the DFT efficient
    ["Bin spacing for fs=1000 Hz, N=1000?", "A 0.1; B 1; C 10; D 1000 Hz", "B", "df = fs/N = 1 Hz."],
```

```
    ["Purpose of low-pass filter?", "A preserve low; B boost all; C randomize; D convert units", "A", "It attenuates higher fr
    ["Convolution length for lengths 3 and 2?", "A 1; B 3; C 4; D 6", "C", "N + M - 1 = 4."],
    ["SNR power formula?", "A 10log10(Ps/Pn); B Ps+Pn; C 20(Ps/Pn); D log difference", "A", "Use 10 log10 for power ratios."],
    ["Why set a random seed?", "A higher fs; B repeatability; C remove noise; D guarantee accuracy", "B", "A fixed seed makes
    ["Best practice for AI code?", "A submit blind; B trust confidence; C run and verify; D remove tests", "C", "Test against
]
wb = Workbook()
ws = wb.active
ws.title = "Signal Processing Quiz"
for row in quiz_rows:
    ws.append(row)
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
    cell.fill = PatternFill("solid", fgColor="244A70")
    cell.alignment = Alignment(wrap_text=True, vertical="top")
for col, width in {"A":42, "B":56, "C":18, "D":52}.items():
    ws.column_dimensions[col].width = width
for row in ws.iter_rows():
    for cell in row:
        cell.alignment = Alignment(wrap_text=True, vertical="top")
ws.freeze_panes = "A2"
wb.save("quiz_spreadsheet.xlsx")
```

Automation is demonstrated by generating the spreadsheet from structured quiz data. In the final submission, include the actual spreadsheet output and verify that each row has the intended answer and explanation.

13. Prompt library

Concept explanation

Explain [TOPIC] twice: first for a beginner using a real-world analogy, then at an advanced engineering level with equations, assumptions, units, and one limitation.

Code generation

Write a self-contained Python demonstration of [TOPIC] using NumPy/SciPy/Matplotlib. Label axes and units. Use deterministic inputs and comment every important step.

Verification-first

Add assertions for at least two known properties of this demonstration. State the expected result before the code. Do not claim execution unless it was run.

Debugging

Review this code for mathematical errors, unit mistakes, array-shape problems, invalid parameters, and misleading plots. Explain each issue and provide the smallest correction.

Plot review

Predict the shape and key features of the plot before executing. After execution, compare the plot to the prediction and explain discrepancies.

Quiz creation

Create 10 multiple-choice questions on [TOPIC], one correct answer each, plausible distractors, and an answer key with a short explanation. Verify calculations.

AI comparison

Answer the same task with clear assumptions, equations, code, and tests. List uncertainties and likely failure modes; do not invent sources or execution results.

Responsible-use review

Identify what in this response is factual, inferred, or uncertain. Suggest independent checks and explain what a student must do themselves.

14. Responsible AI use and limitations

• Use AI as a tutor and drafting aid, not as a substitute for understanding or testing.

• Verify equations against textbooks, course notes, or trusted documentation.

• Run code in a clean environment and preserve the output that demonstrates the result.

• Report errors and corrections honestly. Never claim a test passed unless it was run.

• Do not submit fabricated comparisons, fake citations, or invented experimental results.

• Protect personal information and credentials; do not place API keys or passwords in public repositories.

• Check package versions and API documentation because software interfaces can change.

• Understand the limits of simplified models: real signals may be nonstationary, noisy, nonlinear, or outside ideal theorem assumptions.

• Follow your institution's rules on AI assistance and disclose the tools used when required.

Limitations: the code uses idealized signals and synthetic data. A correct numerical assertion checks only the property it encodes; it does not prove that the whole program is bug-free. Filter behavior depends on design, phase response, boundary conditions, and sampling rate. AI-tool quality changes over time and should be assessed using a reproducible comparison.

15. Final submission checklist

• ■ Beginner and advanced explanations for all six topics

• ■ Python notebook includes six demonstrations and labeled plots

• ■ All code cells run in Google Colab and assertions pass

• ■ Executed notebook saved with outputs

• ■ Known results and verification evidence documented

• ■ Error log contains actual errors observed and fixes made

• ■ At least three AI tools compared using a consistent prompt and recorded evidence

• ■ Two prompting strategies demonstrated and discussed

• ■ Glossary and 10-question quiz include a checked answer key

• ■ Study guide includes formulas and plots

• ■ 5–7 slide presentation prepared

• ■ Quiz spreadsheet generated and opened to verify its contents

• ■ Prompt library saved

• ■ Responsible AI use and limitations explained

• ■ GitHub repository contains README, source, and supporting deliverables

• ■ Final repository link tested and ready to share

Conclusion

This capstone demonstrates a practical way to use AI in signal-processing education: explain the concept, generate a small reproducible program, compare it with known mathematical behavior, and record what was learned. The strongest evidence is not the AI explanation alone, but the executed code, assertions, plots, and honest verification record.

End of report.